

Warm up

We've talked about opaque pointers as a way to prevent clients from accessing the internal fields of an ADT. Consider the following snippet:

```
/* mystery.h */
typedef struct mystery *Mystery_T;
Mystery_T Mystery_new(void);
```

```
/* mystery.c */
struct mystery {
    int *a, *b;
};
```

```
/* mystery client code */
#include "mystery.h"
Mystery_T m = Mystery_new();
```

Is there anything the client code can do to update the fields of m?

(A) `*m = NULL;`

(B) `(int *)m = NULL;`

(C) `*(int **)m = NULL;`

(D) `m[0] = NULL;`

Warm up

```
/* mystery.h */
typedef struct mystery *Mystery_T;
Mystery_T Mystery_new(void);

/* mystery.c */
struct mystery {
    int *a, *b;
};

/* mystery client code */
#include "mystery.h"
Mystery_T m = Mystery_new();
*(int **)m = NULL; /* C is the correct answer */
```

C89 standard Section 3.5.2.1:

A pointer to a structure object, suitably cast, points to its initial member (or if that member is a bit-field, then to the unit in which it resides), and vice versa.

Precept 15: Assignment 4 data structures and algorithms

Polly Ren

COS 217: Introduction to Programming Systems
Spring 2026
Princeton University

March 26, 2026

Overview

- 1 A dynamic array abstract data type
- 2 Directory tree (DT)
- 3 File tree (FT)
- 4 Summary

DynArray interface

- The DynArray ADT is an abstract data type (not an abstract object) that represents a dynamic array of values
- Values are generic (via `void` pointers)
- Similar to a Java ArrayList

```
typedef struct DynArray *DynArray_T;

DynArray_T DynArray_new(size_t uLength);

void DynArray_free(DynArray_T oDynArray);

size_t DynArray_getLength(DynArray_T oDynArray);

void *DynArray_get(DynArray_T oDynArray, size_t uIndex);

void *DynArray_set(DynArray_T oDynArray, size_t uIndex,
                  const void *pvElement);

int DynArray_add(DynArray_T oDynArray, const void *pvElement);
```

DynArray interface (cont.)

```
int DynArray_addAt(DynArray_T oDynArray, size_t uIndex,
                  const void *pvElement);

void *DynArray_removeAt(DynArray_T oDynArray, size_t uIndex);

void DynArray_toArray(DynArray_T oDynArray, void **ppvArray);

void DynArray_map(DynArray_T oDynArray,
                 void (*pfApply)(void *, void *),
                 const void *pvExtra);

void DynArray_sort(DynArray_T oDynArray,
                  int (*pfCompare)(const void *, const void *));

int DynArray_search(DynArray_T oDynArray, void *pvSoughtElement,
                   size_t *puIndex,
                   int (*pfCompare)(const void *, const void *));

int DynArray_bsearch(DynArray_T oDynArray, void *pvSoughtElement,
                    size_t *puIndex,
                    int (*pfCompare)(const void *, const void *));
```

DynArray implementation

- See `dynarray.c` for full code; noteworthy aspects below
- Checking invariants: `assert(DynArray_isValid(oDynArray));`
 - `DynArray_isValid` is a `static` validation function (internally-linked)
 - asserted at the beginning of and before returning from each non-static function
 - Can be inefficient to run in production code, but useful for debugging
 - Can define `NDEBUG` to disable assert statements

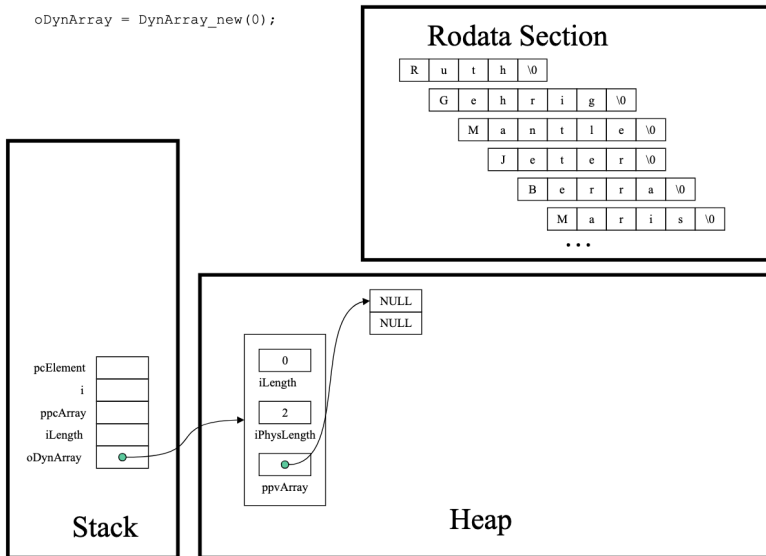
```
#ifndef NDEBUG
static int DynArray_isValid(...) { ... }
#endif
```

- ◊ Compiling with `-DNDEBUG` flag will define `NDEBUG` and thus disable the validation function (i.e., `DynArray_isValid` will not be compiled at all, and asserts will be no-ops)

- `DynArray_grow` is a static function that doubles the physical length (capacity) of the array when needed

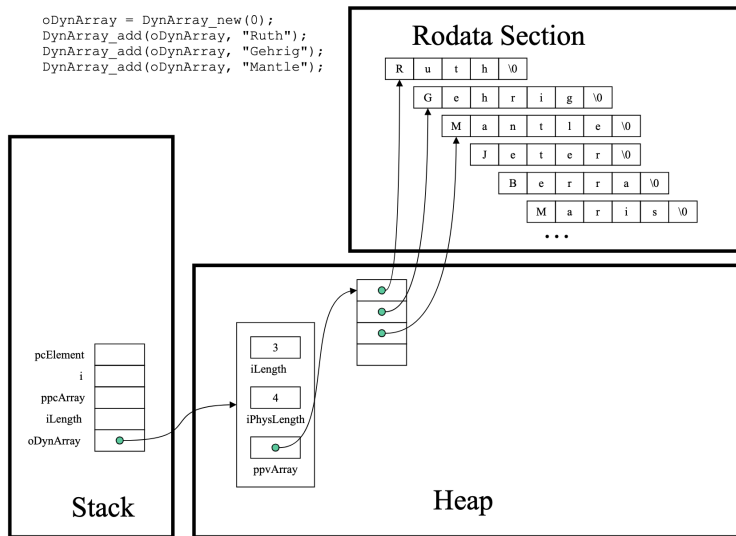
Creating a new DynArray

```
oDynArray = DynArray_new(0);
```



Adding an element to a DynArray

```
oDynArray = DynArray_new(0);  
DynArray_add(oDynArray, "Ruth");  
DynArray_add(oDynArray, "Gehrig");  
DynArray_add(oDynArray, "Mantle");
```



Pros and cons of using DynArray?

- Pros:

- Bounds checking: prevents out-of-bounds access
- Length tracking: can report its length to the client
- Dynamic resizing: can expand as needed to accommodate insertions
 - ◇ Client specifies initial capacity

- Cons:

- Less efficient: access requires function call rather than merely performing pointer arithmetic
- Complexity: not straightforward to work with native types

Overview

- 1 A dynamic array abstract data type
- 2 Directory tree (DT)
- 3 File tree (FT)
- 4 Summary

Recall: Assignment 4 has three parts

- Part 1: Binary Directory Tree (BDT)
 - A hierarchy of directories in which each node may have at most 2 children
 - Task: Use `gdb`, `meminforeport` and `valgrind` to debug and fix memory errors in the provided implementation
- Part 2: Directory Tree (DT)
 - A generalisation of a BDT in which each directory may have an arbitrary number of children
 - Task: write internal validation module to verify validity of the data structure
- Part 3: File Tree (FT)
 - An extension of a DT in which now leaves may represent either an empty directory or a file
 - Task: Implement the file tree interface
 - You get to design the relevant data structures and algorithms

a4def.h

- Contains declarations for different return statuses and declares a boolean type for other parts of Assignment 4 code to use

```
/* Return statuses */
enum {
    SUCCESS,
    INITIALIZATION_ERROR,
    ALREADY_IN_TREE,
    NO_SUCH_PATH, CONFLICTING_PATH, BAD_PATH,
    NOT_A_DIRECTORY, NOT_A_FILE,
    MEMORY_ERROR
};

/* In lieu of a proper boolean datatype */
enum bool { FALSE, TRUE };
/* Make enumeration "feel" more like a builtin type */
typedef enum bool boolean;
```

The DT module

```
/* dt.h */
int DT_insert(const char *pcPath);
boolean DT_contains(const char *pcPath);
int DT_rm(const char *pcPath);
int DT_init(void);
int DT_destroy(void);
char *DT_toString(void);

/* dtGood.c */
/* state variables */
static boolean bIsInitialized; /* is it initialised? */
static Node_T oNRoot;          /* pointer to root node */
static size_t ulCount;          /* number of nodes */
/* ... other implementation code ... */
```

- This is an abstract object!

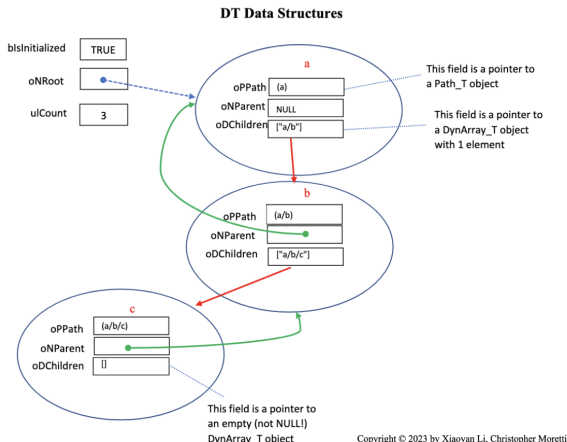
oNRoot is a Node_T, but what is a Node_T?

```
/* nodeDT.h */
typedef struct node *Node_T;

int Node_new(Path_T oPPath, Node_T oNParent, Node_T *poNResult);
size_t Node_free(Node_T oNNode);
Path_T Node_getPath(Node_T oNNode);
boolean Node_hasChild(Node_T oNParent, Path_T oPPath,
                      size_t *pulChildID);
size_t Node_getNumChildren(Node_T oNParent);
int Node_getChild(Node_T oNParent, size_t ulChildID,
                  Node_T *poNResult);
Node_T Node_getParent(Node_T oNNode);
int Node_compare(Node_T oNFirst, Node_T oNSecond);
char *Node_toString(Node_T oNNode);

/* nodeDTGood.c */
/* A node in a DT */
struct node {
    Path_T oPPath;           /* node's absolute path */
    Node_T oNParent;         /* this node's parent */
    DynArray_T oDChildren; /* array of child nodes */
};
```

Visualising nodeDT and DT



- Each `Node_T` is a pointer to a dynamically-allocated ADT
- The Paths and DynArrays that are fields of the Nodes are also dynamically allocated ADTs

The path module

- Used for representing an absolute path in the directory tree (e.g., `"/a/b/c"`)
- Provides functions for handling paths (e.g., `Path_getPathname`, `Path_prefix`, `Path_getSharedPrefixDepth`, etc.)
- This is already implemented for you (in `path.{c, h}`)
- The DT uses functions in the path module to parse and manipulate paths when inserting a node into the directory tree

```
struct path {  
    /* The string representation of the path,  
       which uses '/' as the component delimiter */  
    const char *pcPath;  
    /* The string length of pcPath */  
    size_t ulLength;  
    /* Ordered collection of component strings in path */  
    DynArray_T oDComponents;  
};
```

The checker module

- See `checkerDT.{c, h}`

```
boolean CheckerDT_Node_isValid(Node_T oNNode);
```

- Checks if directory entry (node) is in a valid state

```
boolean CheckerDT_isValid(boolean bIsInitialized, Node_T  
oNRoot, size_t ulCount);
```

- Checks if the entire directory tree is in a valid state

- You will write more tests in Part 2 to catch bugs in the bad DT
 - Add additional checks to the `isValid` functions, in `treeCheck` or in a separate static function that then gets called by one of the previous

Overview

- 1 A dynamic array abstract data type
- 2 Directory tree (DT)
- 3 File tree (FT)
- 4 Summary

File tree interface specification

```
/* ft.h */  
  
int FT_insertDir(const char *pcPath);  
boolean FT_containsDir(const char *pcPath);  
int FT_rmDir(const char *pcPath);  
int FT_insertFile(const char *pcPath, void *pvContents,  
                  size_t ulLength);  
boolean FT_containsFile(const char *pcPath);  
int FT_rmFile(const char *pcPath);  
void *FT_getFileContents(const char *pcPath);  
void *FT_replaceFileContents(const char *pcPath,  
                             void *pvNewContents,  
                             size_t ulNewLength);  
int FT_stat(const char *pcPath, boolean *pbIsFile,  
            size_t *pulSize);  
int FT_init(void);  
int FT_destroy(void);  
char *FT_toString(void);
```

You get to design FT: food for thought

- What state variables are needed for the FT? Same as the DT?
- Do you want to represent directory nodes differently from file nodes?
 - Will probably need to redesign overall structure of program
 - More modular!
- Or, do you want to use the same node structure for both?
 - Could modify current node struct and add more fields
- Or, something else entirely?
- What other modules should the FT interact with?
- How is the algorithm for `FT_insertDir` different from the algorithm for `DT_insert`?
- How is the algorithm for `FT_insertFile` different from the algorithm for `FT_insertDir`?

Be creative :)

Overview

- 1 A dynamic array abstract data type
- 2 Directory tree (DT)
- 3 File tree (FT)
- 4 Summary

Summary

- An overview of Assignment 4 DynArray, DT and related data structures and algorithms
- Dates to remember:
 - Recommended: find an Assignment 4 partner asap
 - Assignment 4: due Tue, 4/7 at 9pm
 - Assignment 4 Quiz: Mon, 4/13 in lecture
- Acknowledgements
 - Most of the code in these slides is adapted from the precept materials, written by Prof. Bob Dondero and Christopher Moretti
 - Can be accessed at <https://www.cs.princeton.edu/courses/archive/spring26/cos217/precepts/w09p2/>